Homework #4

CSED501

Taylor Kessler Faulkner Due: **December 2, 2025**

Name: Aeden Jameson

Instructions: Complete the questions below and upload a PDF containing your written answers to Gradescope.

- Keep your answers within the provided space.
- Answers can be handwritten, or typed and pasted into this PDF.
- Show your work; this makes giving partial credit easier!
- True/False questions can be answered with **True/False**, but explanations can be added if you feel they are needed.
- Numeric answers can be given in fraction or decimal form; if decimal, round to 2 places

Assigned reading (links can also be found in Canvas Week 4 Resources):

- Stackpole, B. (2025) Climate change and machine learning - the good, bad, and unknown, MIT Sloan. (Accessed: 17 November 2025).

Problem 1 **7 pts**: True/False. Mark each statement as True or False and briefly justify your answer.

(a) A Markov Decision Process (MDP) assumes that the future depends on every past action an agent has taken.

(b) The state-value function $V^{\pi}(s)$ represents the expected return starting in state s and following policy π

(c) Softmax exploration assigns higher probability to actions with higher estimated value.

(d) An ϵ-greedy policy always chooses the optimal action with probability 1.

(e) A perceptron with no activation function is equivalent to a linear classifier.

(f) Weight initialization can affect the speed and quality of neural network training.

(g) Batch training computes gradients using the entire dataset at once.

Answer:

a) False. Markov Decision Processes assume the Markov property, which implies that action outcomes only depend on the current state.

b)True. By the defintion given on the slide title "Policies" shown in class week 9.

c) True. As the Q-value increases the numerator of softmax inceases.

d) False. epsilon > 0 is what results in a chance of a random action being picked which would not be optimal.

e) False. The output is a continuous value that doesn't represen a class. Without the activation function the linear function is acting as a regressor.

f) True. Neural netwoks do not cerate convex loss functions therefore there's a possibility of hitting a local optima

g) True. By definition on the slide titled "Batches" show in class week 8.

Problem 2 **14 pts**: Consider a classic RL environment called the Cliff Walking problem. This problem has a 4×12 gridworld environment:

```
. . . . . . . . . . . .
. . . . . . . . . . . .
S . . . . . . . . . . G
! ! ! ! ! ! ! ! ! ! ! !
```

Here, each state is marked by a symbol: S, G, ., or !. The states have the following definitions:

- S is the start state (bottom-left).
- G is the goal state (bottom-right).
- The bottom row of !'s between S and G is the **cliff**.
- Every other state is represented with a period (.)

Consider the following problem settings:

- Stepping into the cliff yields reward -100 and sends the agent back to S.
- All other transitions give reward -1.
- The agent uses an ϵ-greedy policy with $\epsilon = 0.1$.
- The agent uses $\gamma = 1$.

You will analyze how two RL algorithms with different update rules learn in this environment.

(a) Here is the Q-learning update (note the difference from the in-class Q-learning update: $Q(S_t, A_t)$ on the right-hand side is not multiplied by $1 - \alpha$. This is just a slightly different formulation, but α still signifies the weight given to new information):

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\Big].$$

There is another algorithm, SARSA (**S**tate-**A**ction-**R**eward-**S**tate-**A**ction) that uses the following update rule:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\Big].$$

Carefully examine the targets for the two update rules:

$$\text{Q-Learning target: } R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$$

$$\text{SARSA target: } R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$$

In your own words, what quantity is SARSA using to estimate the value of the next state, and how does it differ from the quantity that Q-learning uses?

PROBLEM CONTINUES ON NEXT PAGE

Problem 2 - pts: CONTINUED FROM PREVIOUS PAGE

(b) Suppose the agent receives reward $R_{t+1} = -1$ and arrives in state S_{t+1}. The Q-values in this state are:

Action	Q-value
up	-5
right	-3
left	-4
down	-9

Compute the probability of selecting each action in S_{t+1}, assuming an ϵ-greedy behavior policy with $\epsilon = 0.1$.

(c) Using the Q-values, R_{t+1}, and S_{t+1}, and the computed probabilities from part (b), compute two target values: the Q-learning target value and the *expected* SARSA target value. Remember, an expected value is a weighted sum over all possibilities, each weighted by the probability of the possibility occurring. You do not need to comute the full update, just the targets as defined in part (a).

Answer:

a) The quanity SARSA is using to estimate is the value of the next state-action pair as defined in the SARSA target above. In contrast Q-Learning is using the value of the best possible action in the next state.

b) The behavior policy is ϵ-greedy so the greedy action is right since $Q(\text{right}) = -3$. Given that the probabilites of selecting an action are,

- At random : $\epsilon/4 = 0.025$ for each action
- The greedy action: $P(\text{right}) = (1-\epsilon) + 0.025 = 0.925$

C] The Q-Learning target value is $-1 + 1 \cdot \underbrace{(-3)}_{\max} = -4$

The SARSA target is

$$1 + [0.025(-5) + 0.925(-3) + 0.025(-4) + 0.025(-9)]$$

$$= 1 + (-3.2)$$

$$= -2.2$$

Problem 3 **6 pts**:

In reinforcement learning (RL), an agent interacts with an environment whose dynamics may be **deterministic** (each action produces a fixed next state) or **stochastic** (actions lead to random next states according to a probability distribution).
Consider a gridworld similar to the one shown in class, with no fiery pit. There are walls (##) that are impassable, a start state, and a goal state. An example is shown below of a 4x4 version of this gridworld setting. The agent starts at state S and seeks to reach the goal G with maximum return.

S			
	##		
		##	
			G

There are two versions of this environment:

- **Deterministic**: every action moves the agent exactly as intended, unless blocked.
- **Stochastic**: each intended action succeeds with probability 0.8; with probability 0.2, the agent slips into a random perpendicular direction.

(a) Suppose instead we use a **model-based planning** algorithm such as value iteration. Describe one conceptual reason why planning is easier in the **deterministic** gridworld than in the **stochastic** gridworld.

(b) Conceptually, for Q-learning, how does a stochastic environment affect learning and deployment differently than the randomness induced by ϵ-greedy exploration? Discuss in terms of deployed agent behavior **and** the Q-learning update:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\Big].$$

Answer:

a) Planning is easier in the deterministic world because every action has only one next state to consider, because on each iteration of the Bellman update the expectation sum is 1. In constrast a stochastic world has several states to consider for each action and so more computation is needed.

b) When an agent / robot is deployed in a stochastic environment (e..g one with slippery surfaces) one would set epsilon to 0 or to a very low value so that the agent can use what it has learned including compensating for slippery surfaces. During training / learning that value set so that sufficient exploration can take place (e.g. learning **how to** compensate for slippery surfaces).

When an agent / robot is deployed in said environment with the learned optimal policy that agent or robot needs to be able to sense as many possible states of the state space as possible in order to calculate the optimal return for the term $R_{t+1} + \gamma \max Q(S_{t+1}, a)$. While in a learning environment the Q-learning process is learning leveraging the randomness of exploration reprensented by the epsilon value and the randomness modeling in the state space.

Problem 4 **9 pts**: Consider a simplified robot navigation problem:

- The robot moves (in cardinal directions, North, East, South, and West) in a **gridworld** of size $M \times N$ (rows $\times$ columns).
- At each cell, the robot can face one of **4 orientations**: North, East, South, or West (facing grid walls). This orientation is based on which direction the robot is moving: e.g. if the robot is moving to the right, it is facing right. However, choosing the "South" action will always result in the robot moving South, no matter the orientation.
- The robot can carry **1 object**, which can either be picked up (object present) or not (object absent).

(a) If you want the robot to have all of the information described in the above bullet points, how many possible states are there in the environment for a given $M \times N$ grid? Show your reasoning: what information does each state contain?

(b) What is the **minimum** number of states for a given $M \times N$ grid so that the state space still satisfies the Markov Assumption? Show your reasoning: what information does each state contain?

(c) If the robot's movements are **stochastic** (with some probability of slipping to adjacent cells), does the **state space size** change? Why or why not?

Answer:

a) The number of possible states is M * N * 4 (directions) * 2 (carry state) = 8MN

b) The minimum number of states would be M * N * 2 = 2MN because the orientations could be removed due to it representing the direction the agent moved. However the Markov property depends only on the current state and action.

c) No stochastic movements do not increase the state space because some randomness is being introduced to moving each direction only

Problem 5 **15 pts**: A streaming service tracks customer engagement across three states:

- **State 0:** Active subscriber (watches content regularly)
- **State 1:** Casual subscriber (watches content occasionally)
- **State 2:** Churned (canceled subscription)

The transition probabilities of moving between these states from month to month are modeled as a Markov chain:

$$P = \begin{bmatrix} 0.2 & 0.5 & 0.3 \\ 0.0 & 0.6 & 0.4 \\ 0.1 & 0.0 & 0.9 \end{bmatrix},$$

where $P[i][j]$ represents the probability that a customer in state i moves to state j the following month. For the following questions, you are allowed to use a matrix math calculator to compute your answers, but you must show your work step by step (e.g. for question (a), show the resulting probabilities vector/values for each month (1 and 2).

(a) **Two-Month Churn Probability:** If a customer is an *active subscriber* (state 0) this month, what is the probability that they will have *churned* (state 2) two months from now?

(b) **Long-Term Customer Distribution:** Compute the stationary distribution $\pi = (\pi_0, \pi_1, \pi_2)$ of the Markov chain. Interpret the result in terms of the proportion of customers expected to be active, casual, or churned in the long run.

(c) **Marketing Intervention:** Suppose the company introduces a targeted marketing campaign that reduces the probability of active subscribers churning from 0.3 to 0.1. Conceptually, how would this change the stationary distribution? What would be the expected effect on long-term churn? An intuitive description of how the stationary distribution and long-term churn would change will suffice here: note whether whether each value in the stationary distribution will raise, lower, or stay the same.

Answer:

a) In order to find the probability a subscriber has churned 2 months from now using a Markov chain with the transitions probabilies P we calculate,

P(s0, s1, s2) = P(s0) * P(s1 | s0) * P(s2 | s1)

The inital distribution is P(s0) = [1, 0, 0]. Next we calculate P(s1),

P(s1) = [1, 0, 0]*P = [0.2, 0.5, 0.3]

then P(s2)

P(s2) = [0.2, 0.5, 0.3]*P = [0.07, 0.40, 0.53]

Thus the proabability of churning after two months is 0.53.

b) The stationary condition is

$$P^T \pi = \pi \iff (P^T - I)\pi = 0$$

where $\pi = \begin{bmatrix} \pi_0 \\ \pi_1 \\ \pi_2 \end{bmatrix}$ & $\pi_1 + \pi_2 + \pi_3 = 1$ by the rules of probability

We can write the whole system in matrix form as

$$\begin{bmatrix} -0.8 & 0 & 0.1 \\ 0.5 & -0.4 & 0 \\ 0.3 & 0.4 & -0.1 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} \pi_1 \\ \pi_2 \\ \pi_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

This is an over determined system which can be solved by QR decomposition. Using a online matrix calculator for Q, R & the relation $R\pi = Q^T b$, $b = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$

I got $\boxed{\pi = (0.097, 0.12, 0.78)}$

c)

Since fewer customers are transition to churns I'd expect a lower rate of churn in the long term and consequenctly and increase in active/causal subscribers long term.

Problem 6 **8 pts**: Choose one (1) question to answer:

- Climate data coverage varies greatly across the globe. In 1-2 paragraphs, discuss how biases in data collection or model design might impact vulnerable populations disproportionately.
- In your own words, provide a pro and con argument (each 1 paragraph) for machine learning's potential impact on the environment Your answer should be informed by the assigned reading (Climate change and machine learning - the good, bad, and unknown), but you can reference other sources if desired.

Answer:

Using the Suresh & Guttags framework for understanding biases we can see that the following biases could be present

* Measurement Bias - Wealthy areas will be have the money and technology to collect climate data and measure impact.

* Aggregation Bias - Some wealthier coastal areas where data is collected may end up sharing features with poorer areas underestimate the impact to the poorer community.

* Historical Bias - A poorer coastal area is historically likely to have taken more damage from flooding due to lack of infrasture and so a risk model might treat slightly higher damage due to climate change as typical.

Problem 7 **10 pts**: In 1–2 paragraphs, discuss a problem that you're interested in solving (this could be related to a work problem, a potential personal project, or just something that interests you) that could be solved with RL **or** neural networks. If you propose an RL problem, include an initial reward function you might try and discuss whether you would use a model-based or model-free algorithm. If you propose a neural network problem, discuss a potential dataset and what the output layer of your neural net might be.

Answer:

Context: I work on HBO Max at Warner Brothers Discovery

Problem: Identify actors in shots that comprise a show or movie for searching who is in which scenes, collecting analytics on actors and a relationship graph that identifies who has worked with whom. Assuming one can breakdown a show into shots I'd use a neural network with a input that is a frame / image in a shot and an output layer that is the name of an actor. I'd apply the network to every few frames of the shot. The training data would be images that are labelled with the names of the actors on them. The images should be them in lots of different contexts and image quality if possible.

Problem 8 **10 pts**:

Consider a single-layer neural network with one hidden neuron using a sigmoid activation function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The network takes a 2-dimensional input vector x and has weights $w = [w_0, w_1, w_2]^T = [0.5, 2, -1]^T$. The output of the network is:

$$y = \sigma(w_0 + w_1 x[1] + w_2 x[2])$$

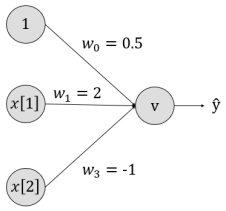

Figure 1: Visualization of described neural network

(a) Compute the output y for the input $x[1] = 1$, $x[2] = 2$. Show your work.

(b) After running input x from part (a) through the network, you find that the true label for x (not the output from the neural net) is 0.8. Using MSE as loss function, perform one round of backpropagation (i.e. update all three weights once). Use the step size $\eta = 0.1$. Show your work. You can use an online calculus tool (or other method) to compute the gradients needed for backpropagation if you so choose, but all other work should be shown step by step. Make sure you explicitly state what the gradient is before plugging in numbers.

Answer:

a) $y = \sigma(w_0 + w_1 x_1 + w_2 x_2)$

$= \sigma(0.5 + 2 \cdot 1 + (-1) 2$

$= \sigma(0.5)$

$= \frac{1}{1 + e^{-0.5}} \approx 0.62.$

b) The MSE is defined as $L = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - t_i)^2$. The new weights are calculated by the rule for gradient descent given by $w_i^{(t+1)} = w_i^{(t)} - \eta \frac{dL(w_i)}{dw_i}$.

First we calculate the gradients by applying the chain rule to L.

$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - t)$, $\frac{\partial \hat{y}}{\partial z} = \sigma'(z) = \hat{y}(1-\hat{y})$

then $\frac{\partial L}{\partial w_j} = 2(\hat{y} - t)\hat{y}(1-\hat{y})\frac{\partial z}{\partial w_j}$. Using a calculator one gets: $\frac{\partial L}{\partial w_0} = \frac{\partial L}{\partial w_1} = -.083$, $\frac{\partial L}{\partial w_2} = .17$

$w_0 = .5 - 0.1(-.083) = .51$

$w_1 = 2 - .1(-.083) = 2.01$

$w_2 = -1 - .1(-.17) = -.98$

Problem 9 **9 pts**: Your usual team of neophytes just found out about reinforcement learning, and they can't wait to deploy the Q-learning algorithm in real life. They were tasked with training a warehouse robot using Q-learning. The robot must pick items from shelves and place them on a conveyor belt. The environment is modeled as a grid, and the robot receives a reward of +10 for each correctly placed item and -1 for every step taken.
The neophytes tell you that they have decided to run RL with the following setup:

- Q-learning with learning rate $\alpha = 1.0$
- Discount factor $\gamma = 0.0$
- ϵ-greedy exploration with $\epsilon = 0.001$ (fixed)

Unfortunately, the robot isn't working very well, and they have come to you for advice.

(a) The robot seems to move randomly and often fails to pick items efficiently. Identify all hyperparameter settings (learning rate, discount factor, and exploration strategy) that could be causing these issues. Propose appropriate adjustments to those hyperparameters that will improve learning and performance. Explain your reasoning. Note: the adjustments can be described as "raise" or "lower" without actual numbers; however, the reason why you choose to lower or raise any hyperparameters must be explained in a sentence.

(b) Suppose the state space is represented as the raw position of the robot in a 100×100 grid and the positions of all items on shelves. The action space is "move up, down, left, right, pick, place." Discuss why Q-learning (or SARSA, or any other similar algorithm) might struggle with this setup. Propose a way to modify the MDP (state space, reward, etc) that might improve Q-learning's performance.

(c) The company notices that when the robot is deployed, it frequently collides with obstacles that weren't modeled in the simulator. Explain why these real-world errors might be taking place.

Answer:

a) Learning rate. Lower this value because per the Q-value update rule with alpha = 1 the terms for the old Q-value are cancelling so all the reward is added to the most recent action and makes convergence unstable.

Discount factor. Raise this value because otherwise only the immediate reward r is added in the update rule and the future rewards are not.

Exploration. Raise this because such a low value results in very little exploration and without sufficient exploration the desired rewards may not be found.

b) Q-learning may struggle in the sense that it will be slow to optimize because the state space is so large. In order to reduce total state space by creating a new state space with the cells grouped in NxN blocks. e.g. 3x3 with the following features in the state space:

* holding or not
* x as defined by the group number
* y as defined by the group number
* binned distance, since the grouping only provides approx locations
* possible direction e.g. N, S, E, W, arrived.

The grouping alone drastically reduces the state space.

c) The collisions in the real world are happening because the RL simulation didn't contain a penality for states that are not desirable to collide with.

Problem 10 **9 pts**: Now your neophytes have learned about neural networks, and they want to apply them to any and every problem. They have been tasked with predicting the time until failure (in hours) for a set of industrial machines using sensor data. Each machine has features such as vibration level, temperature, operating hours, and load.
They come to you with the following plan:

(a) They'll standardize the features (and labels if needed) to put all numbers in a similar range.

(b) Because this is a regression problem, they will not use activation functions on any of their neurons.

(c) They'll run training for 1 epoch on a dataset of 20,000 machine readings.

For each step of the team's proposal, write 1–2 sentences arguing either:

- That the step is appropriate and why
- That the step is inappropriate, a suggestion for what to do instead and why

Answer:

a) Appropriate. Standarizing the data helps with convergence and thus stabilizes learning.

b) Inappropriate. The acitivation functions in the hidden layers aid in learning non-linear relationships as demonstrated by the x-or dataset.

c) Inappropriate. The chances the network can learn the function over 20k samples in 1 epoch is unlikely.

Problem 11 **3 pts**: How many hours do you estimate you spent on this assignment? (graded on completion of (a), (b), and (c))

(a) Coding:

(b) Written:

(c) Reading:

Answer:

a) Coding: 3.5 hrs

b) Written: 7 hrs

c) Reading: 10 hrs

Problem 12 **Gratitude pts**: Any feedback you would like to share (optional)?

Answer:

Nothing this time.